UNIVERSITY OF LONDON

ST 2195 COURSEWORK 2025 REPORT

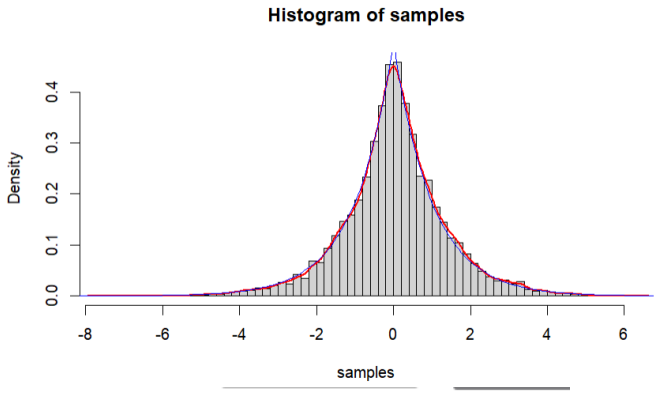
Done by: Ishwar Kumar Ganesh

Student No:

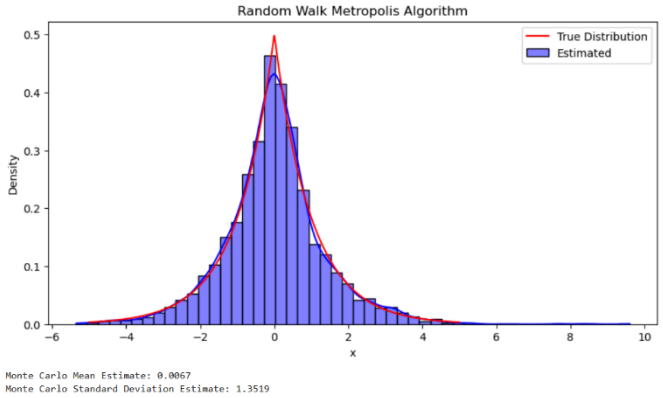
Table of Contents

Question 1a

The random walk Metropolis algorithm was executed 10,000 times. The initial value x0 of 0 and a standard deviation of 1 is set. The probability function of $f(x) = \frac{1}{2}\exp(-|x|)$ and a for loop that iteratively generates random samples from a normal distribution is created. The samples calculation involves dividing the probability density of the new sample by the density of the previously accepted sample. These samples are put against an evaluation ratio and if it exceeds a random value between 0 and 1, the new sample is kept. Otherwise, the previous sample is kept. Following 10,000 iterations, it is plotted onto a histogram with a kernel density plot for visual analysis. The histogram reveals the values that are most likely to occur. The plots are as follows: (**R on left, Python on right**)



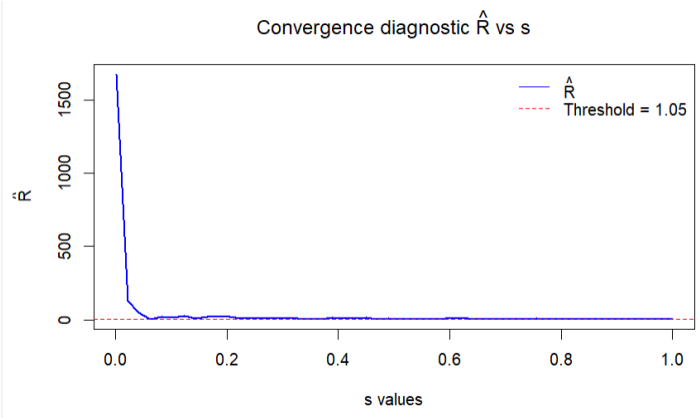

```
Mean Estimate: 0.02974851
S.D Estimate: 1.334803
```

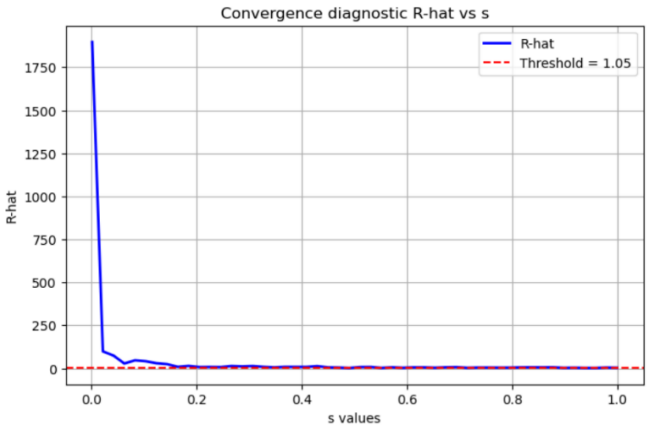



```
Monte Carlo Mean Estimate: 0.0067
Monte Carlo Standard Deviation Estimate: 1.3519
```

Question 1b

We execute the Metropolis-Hastings algorithm to evaluate the convergence attributes of a Markov Chain Monte Carlo (***MCMC***) process. The function compute Rhat determines the Gelman-Rubin diagnostic (R-hat) to evaluate the convergence of several chains. The script runs four independent chains with various proposal standard deviations (***s***) ranging from 0.001 to 1, calculates the corresponding R-hat values, and generates the plots. The graph illustrates how changes in ***s*** influence convergence, with a red dashed line marking the threshold of 1.05, below which convergence is deemed acceptable. (**R on left, Python on right**)



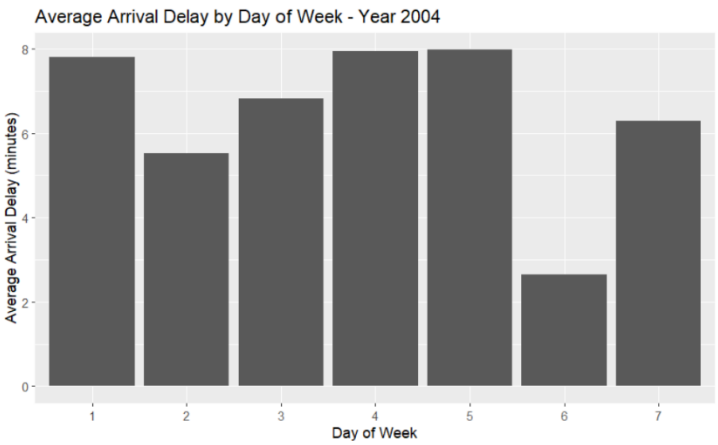


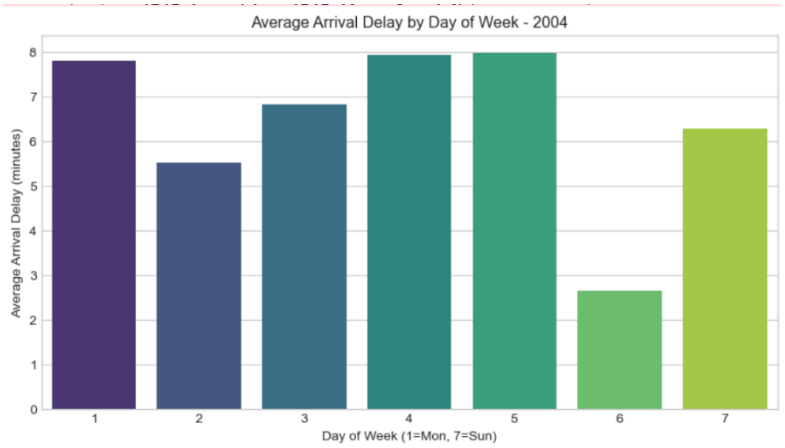
Question 2

I chose to use the years 2004-2008 for data analysis. Due to the large size of the files, some data cleaning was necessary. I checked if the data had the necessary columns such as "DayOfWeek", "CRSDepTime" and "ArrDelay". A warning is displayed if there are any missing columns. I also had to convert “CRSDepTime” to Hour of the day by dividing it by 100 so for example, 1420 will display as 14. This sorts the exact times of flights by the hour of the day. Any rows with ‘NA’ in “ArrDelay” were filtered out.

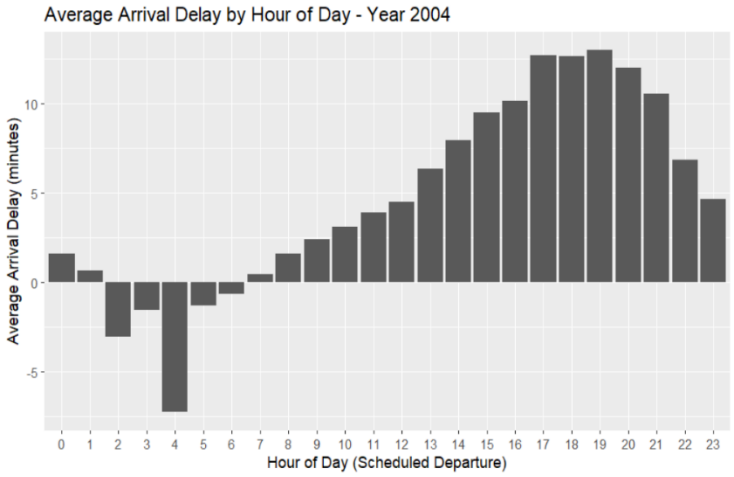
Question 2a

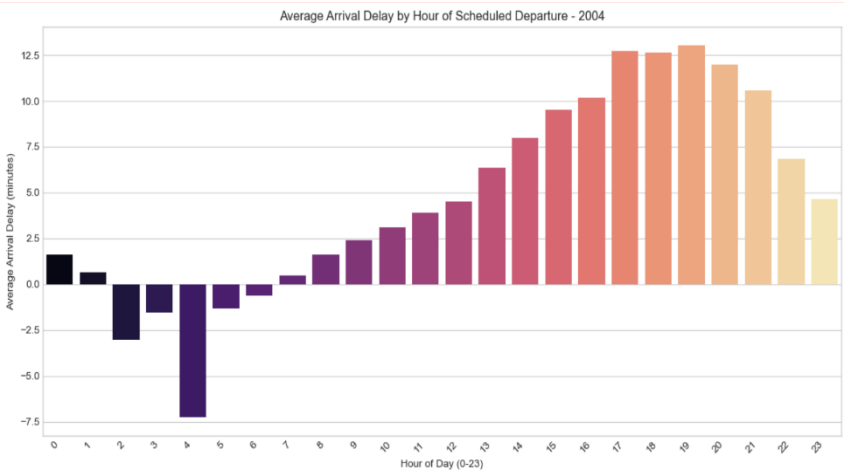
I did an analysis of flight arrival delays over multiple years (2004–2008) using data from yearly CSV files. After loading the datasets for the years 2004 – 2008, I ensured that the dataset contains the required columns—"**DayOfWeek”**, “**CRSDepTime”** (scheduled departure time), and “**ArrDelay”** (arrival delay). The “**CRSDepTime”** column is transformed to extract the hour of departure, and rows with missing “**ArrDelay”** values are removed. Next, I calculated the average and median arrival delays grouped by “**DayOfWeek”** and departure hour. In terms of the fewest delays, it determines which day and hour perform the best. Key findings, including the ideal day and hour with the fewest delays, are printed on the console and the results are kept in a list for every year. I also created a number of visualizations, such as bar charts and boxplots, to show delay trends by hour of the day and day of the week. Lastly, a summary report that compares the optimal travel times for each of the examined years is printed. This offers insights into past trends in flight delays, showing the best performing best day and hour to travel per year. Due to space constraints, a few examples are provided. (**R on left, Python on right**).



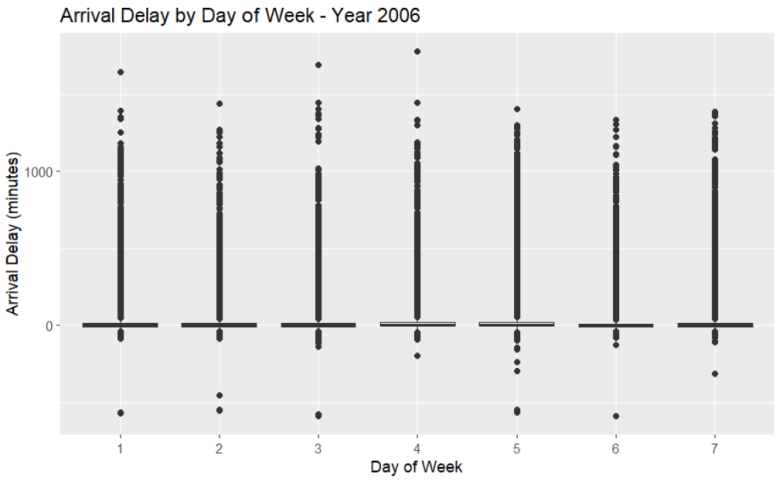


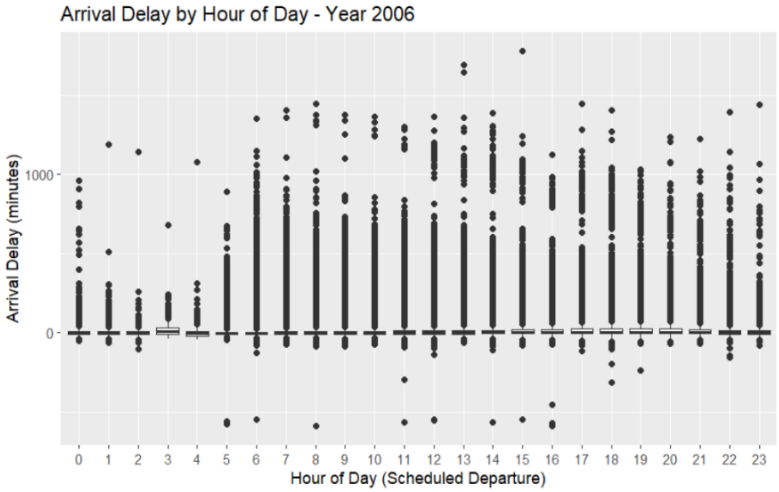
The bar chart above shows the average arrival delay by day of the week. This is conducted for every year and 2004 is chosen as an example. The days are numbered 1-7 with 1 being Monday and 7 for Sunday as seen on the Python chart on the right. I also continued and dove deeper into the average delays per hour for every year to find out the best timing to travel to have the least amount of delays. The year 2004 is chosen as an example. (**R on left, Python on right**).





In addition, a scatter-plot with the delays is also created in R for the day and hour of every year to show the frequency of flights being delayed per year. I have chosen the year 2006 as an example. The negative values of delays show the flight was actually **early** instead of delayed. This is displayed throughout the visualisations provided.





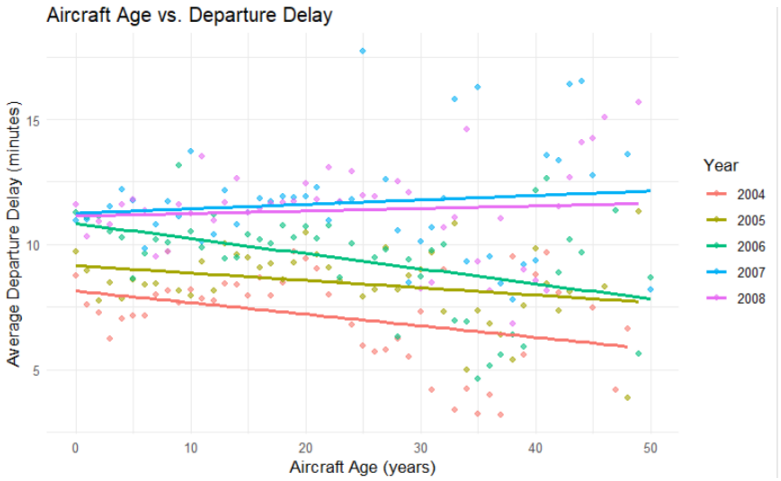
Finally, I also had a created a summary letting us know the day and hour that the least amount of delays was experienced per year. As seen earlier, the days are number 1-7 with Monday being 1 and Sunday being 7. (**R on left, Python on right**). The Python summary on the right is the result for the year 2008 as an example. Both R and Python have summaries for every year.

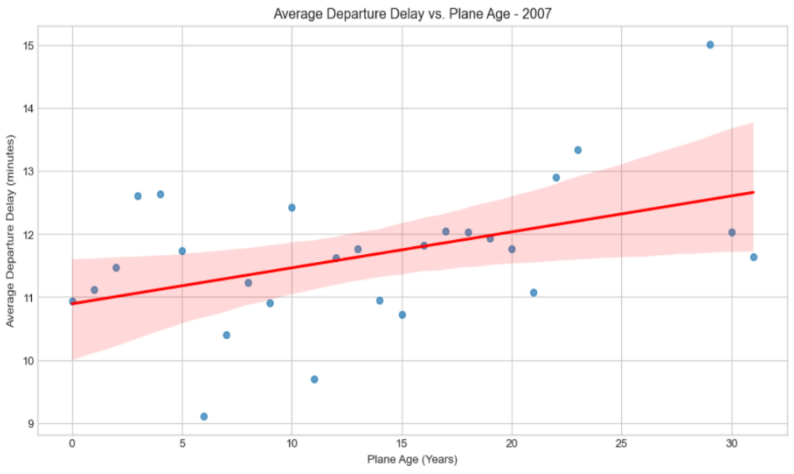
```
--- Summary of Best Times and Days Across Years (based on average delay) ---
Year: 2004 - Best Day: 6 , Best Hour: 4
Year: 2005 - Best Day: 6 , Best Hour: 5
Year: 2006 - Best Day: 6 , Best Hour: 5
Year: 2007 - Best Day: 6 , Best Hour: 3
Year: 2008 - Best Day: 6 , Best Hour: 4
```

```
-- Results --
Best Day of Week (Avg Delay): Day 6 (7.51 min avg delay)
Best Day of Week (Median Delay): Day 6 (-2.00 min median delay)
Best Hour of Day (Avg Delay): Hour 04:00 (-0.21 min avg delay)
Best Hour of Day (Median Delay): Hour 03:00 (-7.00 min median delay)
```

Question 2b

For question 2b, we are tasked with finding if older planes lead to more delays. The aircraft dataset, 'plane-data.csv' is loaded which has information on aircraft manufacturing details, including the tail number 'tailnum' and year of manufacture 'year'. Some columns are renamed in planes dataset in order to merge properly with the other yearly dataset I already loaded. For example, 'tailnum' in the plane data is renamed to 'TailNum' as per the yearly dataset. This allows the merge to occur with matching columns and not get errors. The dataset is cleaned to remove some values (e.g., aircraft built before 1950 or after the analysed period). After merging, a new variable, 'AircraftAge', is computed as: 'AircraftAge'='FlightYear'−'YearBuilt'. The purpose of this is to get the age of the aircraft in a numeric value at the point of departure. Only aircrafts with a valid 'AircraftAge' (between 0 and 50 years) are retained to exclude erroneous data. The scatter plots below show the relationship between the plan's age and the amount of delays. (**R on left, Python on right**). 2007 is chosen as an example for the Python script.





I also created a visualisation in the form of bar graphs to display the percentage of flights being delayed by year. It categorizes aircraft into age groups (0-5, 5-10, 10-15, 15-20, 20-30, 30-50 years) and visualizes the distribution of departure delays for each group per year. This delay function takes into account delays of 15 minutes or more. 2007 is used as an example.

The following console output is generated in R after the running of the code:

```
Call:
lm(formula = DepDelay ~ AircraftAge, data = flights)

Residuals:
     Min       1Q   Median       3Q      Max
-1209.62   -13.73    -9.87    -0.92  2591.16

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 9.6057840  0.0112970  850.30   <2e-16 ***
AircraftAge 0.0156011  0.0009497   16.43   <2e-16 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 33.37 on 22970164 degrees of freedom
  (158783 observations deleted due to missingness)
Multiple R-squared:  1.175e-05, Adjusted R-squared:  1.17e-05
F-statistic: 269.8 on 1 and 22970164 DF,  p-value: < 2.2e-16
```

The residuals summarize how much the predicted departure delays deviate from the actual departure delays. The median residual is -9.87, meaning that half of the predicted delays are off by about 9.87 minutes. The minimum residual (-1209.62) and maximum residual (2591.16) indicate extreme outliers where predictions were very different from actual values.

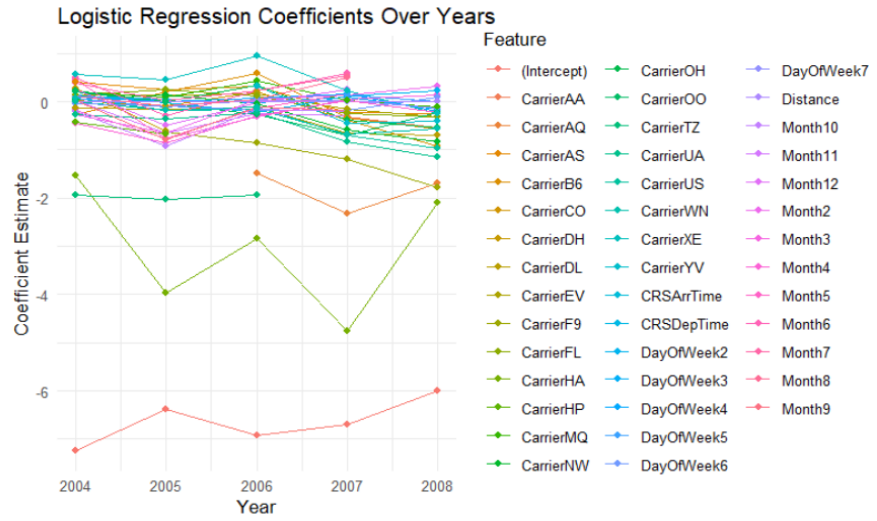
For the coefficients table, each row represents a variable in the regression equation. For the Intercept row, this is the expected departure delay (in minutes) **when the aircraft age is 0** (i.e., a brand-new aircraft). This shows that even the newest planes experience an average departure delay of about **9.61 minutes**. The AircraftAge row represents the effect of aircraft age on departure delay. For each additional year in aircraft age, the average departure delay increases by 0.016 minutes (about 0.94 seconds). While statistically significant (p-value < 2.2e-16), this effect is hardly noticeable in practice. The standard error measures the accuracy of the coefficient estimate and a smaller standard error indicates a more precise estimation. The t-value measures the strength of the relationship between “AircraftAge” and “DepDelay”. The large t-value suggests that the relationship is unlikely to be due to random chance. Since the p-value is much smaller than 0.05, we reject the null hypothesis and conclude that aircraft age does have a statistically significant effect on departure delay.

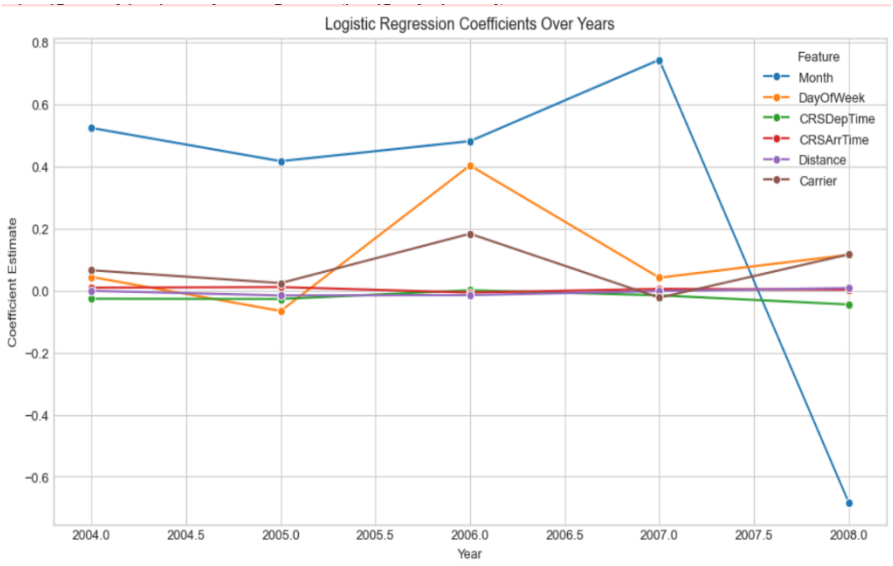
The null hypothesis states that aircraft age has no effect on departure delays. Although the p-value is small, we can conclude that aircraft age does have a statistically significant effect on departure delay.vResidual standard error indicates that sthe actual departure delays vary by **about ±33.37 minutes** around the regression line. Since the aircraft age effect is **only 0.016 minutes per year**, this suggests that **other factors contribute much more** to departure delays. The very low R-squared value also confirms that aircraft age has little to no predictive power for departure delays. Overall, since the **effect size is tiny and R-squared is extremely low**, aircraft age is **not a meaningful predictor** of flight delays in a practical sense.

Question 2c

I use specific columns such as "Diverted", "Month", "DayOfWeek", "CRSDepTime", "CRSArrTime", "Distance", "UniqueCarrier" to perform analysis. To efficiently load and process the large datasets, I had to read each year's flight data in chunks of 100,000 rows to manage memory usage. "CRSDepTime" and "CRSArrTime" are converted to uint16 (unsigned 16-bit integers) to optimise memory usage. Missing values in the "Diverted" column are removed to ensure a clean dataset for model training. For each year, the dataset is filtered to only have the flights from that year. The model coefficients from each year are collected into a dataframe and reshaped to facilitate visualization. A line plot is generated to show the trend of each coefficient over the years.

Positive coefficients indicate that an increase in the feature value increases the likelihood of diversion while negative coefficients indicate that an increase in the feature value decreases the likelihood of diversion. By conducting this analysis, it helps to identify trends over the years, such as whether flight distance, time of departure, or airline carrier had a growing or diminishing effect on diversions. The following visualisations are created by running the code. (**R above, Python below**).





The X-Axis (Year) represents the years from 2004 to 2008, indicating how the coefficients change over time. The Y-Axis (Coefficient Estimate), represents the magnitude and direction of the influence each feature has on the likelihood of flight diversion.

Features such as “CRSDepTime” (Scheduled Departure Time) and “CRSArrTime” (Scheduled Arrival Time) remain fairly stable alongside “Distance”. Most ‘Days of the Week’ such as “DayOfWeek3”, “DayOfWeek5” remain somewhat stable but there are some ‘Days of the Week’ such as “DayOfWeek2” & “DayOfWeek7” have some fluctuations about them suggesting that certain days became more or less prone to diversions over the years.